FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-013 Dataset Inspector Mini-Lab

Integrated C3-00 build | teacher setup -> independent implementation -> validation -> recovery -> evidence

Artifact	C3_013_DATASET_INSPECTOR_MINI_LAB_RC1
Status	2026 - LEARNER EDITION
Teaching standard	M01 beginner-first teacher loop; book remains sufficient without video.
Bridge boundary	Course 2B competencies may be assumed; all new engineering concepts still start from first principles.
Brand	Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6 | AMOLED #000

House rule: understand -> follow once -> see expected result -> break/fix -> do a changed version -> validate -> explain -> save evidence. A running command alone never counts as mastery.

Architecture picture

SCENARIO

You inherit a drop-zone CSV utility. Operations needs a small Dataset Inspector that can be run by a human today and orchestrated later. It must fail clearly on bad input, leave deterministic profile evidence, be testable, have reviewable Git history, and be rerunnable.

SETUP - FOLLOW ONCE

Unzip `C3_013_DATASET_INSPECTOR_MINI_LAB_STARTER_RC1.zip` into a fresh Linux-native Git project. Create/recreate `.venv`, install requirements, run the placeholder test, inspect starter data, then create a feature branch. Do not implement from review material.

REQUIRED COMMAND CONTRACT

```
python -m dataset_inspector.cli INPUT.csv --output out/profile.json
# optional config: --log-level; course precedence = CLI > C3_LOG_LEVEL > INFO
```

REQUIRED OUTPUT FIELDS

Profile JSON must include input path/name, file bytes, data-row count, ordered columns, missing required columns, and SHA-256 of the input. Use deterministic key ordering/format for fields under your control. If your output includes runtime timestamps, do not claim byte-identical determinism; compare business/profile fields separately.

TEACHER CHECK BEFORE INDEPENDENT BUILD

Before coding, write the contract: expected input grain, required columns, output grain, failure codes, publish rule, rerun rule, validation controls and evidence location.

Independent staged build

1 - INPUT BOUNDARY

Reject nonexistent, empty, unreadable or non-CSV input as appropriate. Do not publish a success profile.

2 - REUSABLE CORE

Implement file/hash/schema/row inspection in callable core functions. CLI parsing must not be the only way to use the logic.

3 - LOGGING AND EXCEPTIONS

Log safe run/stage context. Known validation failures are clear/non-zero. Unexpected exceptions keep diagnostic context at application boundary.

4 - TESTS

At least 6 meaningful pytest tests: valid case, missing required column, empty input, missing path, deterministic/repeat behavior, plus one changed edge case. Use `tmp_path` where useful.

5 - GIT HANDOFF

Feature branch, coherent commits, `.gitignore`, dependency declaration, README, RUNBOOK. No secret/disposable environment/pycache tracked.

6 - RERUN PROOF

Run unchanged input twice. Prove the intended profile does not accumulate/duplicate. Save comparison evidence.

7 - DOCKER PROOF

If Docker available: build image, run `--help`, then mounted input/output run. If unavailable: record blocker + Dockerfile/commands/expected result; runtime success remains explicitly NOT VERIFIED.

FAILURE / RECOVERY DRILL

Deliberately remove/rename one required column. Save failure evidence. Restore/change input safely and rerun. Previous known-good output must not be silently replaced by a failed/partial artifact.

COMPETENCY GATE - NO AVERAGE

Mini-Lab PASS requires ALL of the following: Contract & grain PASS; filesystem/environment PASS; package/CLI boundary PASS; configuration/secret boundary PASS; logging/failure PASS; testing PASS; rerun/reproducibility PASS; Git/runbook handoff PASS. Docker-runtime competency is PASS if real execution is verified, or FALLBACK-VERIFIED if the approved design-only path was required and blocker evidence is honest. Any core FAIL blocks the Mini-Lab.

CRITICAL FAILURES

Secret disclosure; fabricated runtime evidence; destructive unsafe command/data loss; silent success after contract failure; copying protected review as solution; overwriting/altering the frozen first attempt; unresolved wrong-result validation gap.

EVIDENCE BUNDLE

Contract, source, tests, pytest output, run commands + exit codes, profile outputs, run1/run2 comparison, success/failure logs, Git log/diff summary, README, RUNBOOK, Docker evidence/fallback, failure/recovery drill, and 2-minute interview defense.